

RichlyBook 2.0 — New Sale Backend / Data Design

New Sale + Custom Item + Partial Payment
Design only — no code, migrations, or UI implemented in this document
Date: 8 October 2026

Goal: Vyapar-like counter completion path while *reusing* orders → payments → sales, inventory, and journals — not a second sales system.

1. Design principles

- 1. **One domain spine** — every counter sale is still an orders row that can become a sales row.
- 2. **Server-authoritative money** — totals, paid, due, stock, journals computed in RPCs.
- 3. **Extend, don't fork** — new RPC + small schema extensions; no parallel counter_sales tables.
- 4. **Wholesale path stays** — assisted order → approve → pack → deliver unchanged.
- 5. **Catalogue + custom lines coexist** on the same order.
- 6. **Partial pay half-exists** — reuse cash_collected_amount / online_collected_amount + PAYMENT_PENDING; Admin full-pay RPC is the counter gap.

2. Target owner flow (data view)

```
Owner New Sale (UI)
→ admin_complete_counter_sale (single transaction)
  |
  ├── resolve shop (existing | quick-create | Walk-in system shop)
  ├── write order + order_lines (catalogue + custom)
  ├── reserve/consume stock ONLY for catalogue lines
  ├── record collection (0..total) → payments
  ├── create sale + sale_items + invoice #
  └── optional immediate journal post for Books
```

Wholesale assisted create (place_assisted_order) remains for salesman / long lifecycle.

3. New Sale — order model

3.1 Fast-path on existing orders

Decision	Choice
Storage	Same orders / order_lines / payments / sales
Discriminator	New order_source value, e.g. COUNTER_SALE
Lifecycle	Server short-circuits to complete in one RPC (no OTP / delivery required)
Actor	Admin (is_admin()); created_by_profile_id = owner/operator

Why not a new counter_sales table? Duplicates invoice numbering, outstanding, day book, convert, refund. Audit forbids a second sale system.

3.2 Status machine for counter sales

Step	Counter path (one TX)	Wholesale path (unchanged)
Create	Insert order	place_assisted_order → awaiting confirmation
Stock	Reserve+consume catalogue lines in finalize TX	Approve → reserve; convert → consume
Payment	Record 0..total	admin_mark_payment_received / COD
Sale	Always create sales when finalized	Convert only if DELIVERED + PAID today

Credit / partial: Finalized counter sale *must* create sales even when `payment.status ∈ {UNPAID, PAYMENT_PENDING}`. That is the main gate change vs today's convert RPC.

3.3 Draft

Option	Recommendation
A. No DB draft — one-shot finalize	v1
B. Counter draft status + upsert RPC	Phase 2 if resume needed

v1: local UI draft only (like Sales PWA), not persisted draft rows.

3.4 Walk-in customer

`orders.shop_id` is **NOT NULL** — do not nullable it.

Option	Recommendation
System Walk-in shop (one per business)	Preferred — flag e.g. <code>shops.is_walk_in</code>
Quick-create real shop from sale	Supported as alternate input on same RPC
Party snapshot on order	Only if per-bill walk-in name/phone needed later

Walk-in ledger aggregates as one bucket (acceptable for retail). Optional later: display-only

`orders.walk_in_label`.

4. Custom Item — line model

4.1 Constraint today

- `order_lines.sku_id` uuid NOT NULL
- Unique (`order_id`, `sku_id`)
- Inventory / pricing RPCs assume real SKUs
- `sale_items.sku_id` **is already nullable** — sale snapshot layer is ready

4.2 Recommended: line_kind + nullable sku_id

```
order_lines
+ line_kind CHECK IN ('CATALOGUE','CUSTOM') DEFAULT 'CATALOGUE'
+ sku_id uuid NULL -- was NOT NULL
+ line_discount numeric(12,2) DEFAULT 0 -- optional v1
CHECK (
  (line_kind = 'CATALOGUE' AND sku_id IS NOT NULL)
  OR (line_kind = 'CUSTOM' AND sku_id IS NULL
      AND product_name_snapshot <> ''
      AND selling_unit_snapshot <> '')
)
DROP unique (order_id, sku_id)
CREATE UNIQUE INDEX (order_id, sku_id) WHERE sku_id IS NOT NULL
```

Custom payload: { kind:'CUSTOM', name, unit, quantity, unitPrice, discount? }

Catalogue: { kind:'CATALOGUE', skuId, quantity, unitPrice? }

Snapshots already exist — custom fills product_name_snapshot, selling_unit_snapshot, agreed_unit_price, line_total. Codes can be 'CUSTOM'.

4.3 Rejected alternatives

Alternative	Why reject
Ephemeral SKU per custom line	Pollutes catalogue / MOQ / stock
Shared "MISC" SKU	Breaks uniqueness, analytics, stock
Separate custom_order_lines	Dual reads on invoice, convert, ledger

4.4 Inventory rules

Line kind	Reserve	Consume
CATALOGUE	Yes	Yes
CUSTOM	No	No

Update reserve/consume helpers to skip line_kind = CUSTOM.

4.5 Pricing / override policy

Today server ignores client unit price on assisted place/replace.

Line	Price rule
CATALOGUE	Default = effective trade price; allow override on counter for admin; persist in agreed_unit_price
CUSTOM	Agreed price is authoritative (no catalogue resolve)

Wholesale RPCs can keep authoritative trade pricing unless an explicit override flag is added later.

4.6 Convert / invoice / discounts

- Finalize / convert copies custom lines into `sale_items` with `sku_id = NULL`.
- Invoice print uses snapshots — works if UI shows custom name.
- v1 bill discount via existing `orders.adjustments`; keep `total = subtotal + adjustments`.

5. Partial Payment — payment model

5.1 Reuse existing

Piece	Role
<code>payments.amount</code>	Order total (due basis)
<code>cash_collected_amount / online_collected_amount</code>	Split collections
<code>UNPAID → PAYMENT_PENDING → PAID</code>	Partial already used by delivery COD
<code>payment_events</code>	Audit trail
Customer ledger residual	<code>total - cash - online (already)</code>

5.2 Gap

`admin_mark_payment_received` always sets full `PAID` for `order.total`; Admin UI does not pass method/amount.

5.3 New RPC: `admin_record_order_collection`

Input: `p_order_id`, `p_amount` ($>0, \leq \text{remaining}$), `p_method` (`CASH|UPI|BANK|OTHER`), `p_note`?

1. Ensure `payments` row with `amount = orders.total`.
2. `remaining = amount - cash - online`; reject if overpay.
3. Add to cash or online bucket by method.
4. Status: `0 → UNPAID`; partial `→ PAYMENT_PENDING`; full `→ PAID + paid_at`.
5. Append `payment_events`.
6. Keep one payment row per order (`payments_unique_order`).

Credit (₹0 at finalize): payment row `UNPAID`, collected 0 — sale still created.

Full pay: same RPC with `p_amount = remaining`.

5.4 Sale vs payment status

Scenario	<code>payment.status</code>	Create sale?	Outstanding
Full cash/UPI	<code>PAID</code>	Yes	0
Partial	<code>PAYMENT_PENDING</code>	Yes (counter)	remaining
Credit (0 paid)	<code>UNPAID</code>	Yes (counter)	total
Wholesale convert	Requires <code>PAID</code> today	Unchanged unless product decides	—

Credit = unpaid residual — do not invent a `CREDIT` payment status.

5.5 sales_payments + journals

- At finalize: `sales_payments.amount` = cash + online (may be < total); status PAID / PARTIAL / UNPAID.
- Later collections update payments only; ledger/day book already key off payments.
- Sale journal: Dr AR / Cr Sales for *full invoice total*.
- Collection journal: Dr Cash/Bank / Cr AR for *delta collected*.
- Prefer post-in-RPC for counter finalize so Books feel immediate.

5.6 Method mapping

Owner	Bucket	collection_method
Cash	cash_collected	CASH_ON_DELIVERY or new CASH_COUNTER
UPI	online_collected	UPI_ON_DELIVERY or UPI_COUNTER
Bank	online_collected	bank-ish / OTHER
Credit	neither (amount 0)	document as unpaid residual

6. Proposed RPC surface (v1)

RPC	Purpose
admin_complete_counter_sale	Atomic: shop, lines, stock, payment 0..total, sale+invoice
admin_record_order_collection	Later collections on open due
Reuse	Invoice allocator inside complete; refund = existing full refund RPC
Unchanged	place_assisted_order, wholesale advance/pack/deliver

admin_complete_counter_sale input shape

```
shop: { shopId } | { walkIn: true } | { quickCreate: { name, mobile, ... } }
lines: [
  { kind:'CATALOGUE', skuId, quantity, unitPrice? },
  { kind:'CUSTOM', name, unit, quantity, unitPrice, discount? }
]
payment: { amountPaid, method } -- method ignored if amountPaid = 0
billDiscount? -- → orders.adjustments
notes?
```

Output: orderId, saleId, invoiceNumber, subtotal, discount, total, amountPaid, amountDue, paymentStatus

7. Integrity rules (encode in SQL)

1. `amountPaid + amountDue = total` (within rounding).
2. `cash_collected + online_collected ≤ payments.amount`.
3. Custom lines never touch `inventory_balances`.

4. Catalogue lines fail finalize if insufficient stock.
5. Optional idempotency key (p_client_request_id) against double-submit.
6. Money numeric(12,2); qty numeric(12,3).
7. FOR UPDATE on order + payment.
8. No client-trusted grand total — server recomputes from lines + adjustments.

8. Impact on existing features

Feature	Impact
Customer ledger / receivables	Works if residual formula unchanged
Day book	Verify credit sale appears with due; partial as collection
Sales register	New rows; optional filter by source
Invoice print	Snapshots + paid/due
Assisted wholesale	Untouched if gated by order_source
Commission	Today ties to SALESMAN_ASSISTED — counter skips unless decided
GST on lines	Out of v1; catalogue HSN/rate only

9. Explicit non-goals (v1)

- Partial line returns / RMA
- Stored PDF / WhatsApp
- Customer bill-photo OCR
- Barcode / MRP
- Changing wholesale price-overwrite policy globally
- Multi-payment history table beyond payment_events

10. Migration sequence (when implementing)

1. **Schema:** line_kind, nullable sku_id, partial unique index; optional shops.is_walk_in; order_source value.
2. **Inventory helpers:** skip custom lines.
3. **admin_record_order_collection** + tests vs COD invariants.
4. **admin_complete_counter_sale** + tests (catalogue, custom, mix, full/partial/credit, stock, idempotency).
5. **Relax sale gate** for counter source only.
6. **Journal:** post sale + collection for that id.
7. **Admin UI last** — bind only to these RPCs.

11. Decision checklist (lock before coding)

#	Decision	Recommendation
D1	Counter storage	Same orders/sales + COUNTER_SALE source
D2	Custom lines	Nullable sku_id + line_kind
D3	Draft in DB	No for v1
D4	Walk-in	One system shop + optional quick-create
D5	Catalogue price override on counter	Allowed for admin
D6	Credit/partial can create sale	Yes for counter; wholesale convert stays PAID
D7	Collection RPC	admin_record_order_collection; reuse cash/online columns
D8	Journals on finalize	Prefer post-in-RPC for that sale/collection

12. Success criteria (backend)

A single RPC call can:

1. Sell 2 catalogue SKUs + 1 custom "Special Grocery Item".
2. Accept ₹500 of ₹1800 (UPI).
3. Create invoice/sale with due ₹1300.
4. Reduce stock only for catalogue SKUs.
5. Show correct outstanding on customer (or Walk-in) ledger.
6. Not break assisted wholesale orders.

Next implementation step (later): lock D1–D8 → schema + two RPCs + db-tests only (no UI) → Admin New Sale UI against `admin_complete_counter_sale`.